

a) Problem: Analog to Digital Signal Conversion

Logical Solution:

- Sampling the time domain with specific period which is attained from sampling frequency.
- It creates discrete time signal.
- Then quantized the sine signal, which makes the signal as discrete valued signal
- Digital signal is both a discrete time signal and discrete valued signal.

Source Code:

```
#x(t)=cos(2*pi*f*t) plotting from t(-1,1)
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
tmin=-1
```

```
tmax=1
```

```
num_t=1000
```

```
t=np.linspace(tmin,tmax,num_t)
```

```
#print(t)
```

```
f=4
```

```
xt=np.cos(2*np.pi*f*t)
```

```
plt.plot(t,xt)
```

```
plt.xlabel("t")
```

```
plt.ylabel("Amplitude")
```

```
plt.title("Plot of cosine function")
```

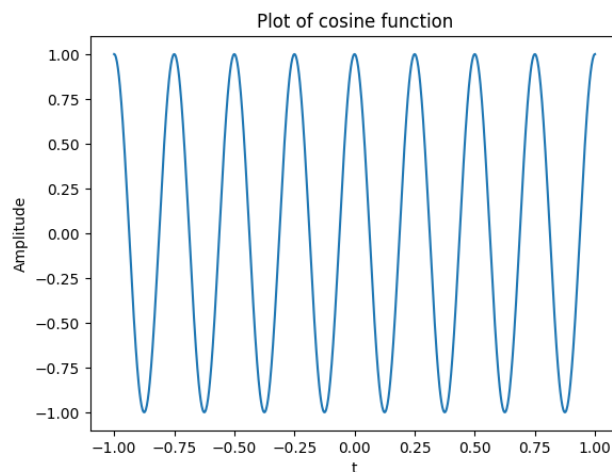


Figure-Sine Signal

```
#sampling
```

```
Fs=30
```

```
Ts=1/Fs
```

```
pulse_train=np.arange(tmin,tmax, Ts)
```

```
#print(pulse_train)
```

```
xt_sampled=np.cos(2*np.pi*f*pulse_train)
```

```
plt.stem(pulse_train,xt_sampled)
```

```
plt.plot(t,xt,'r')
```

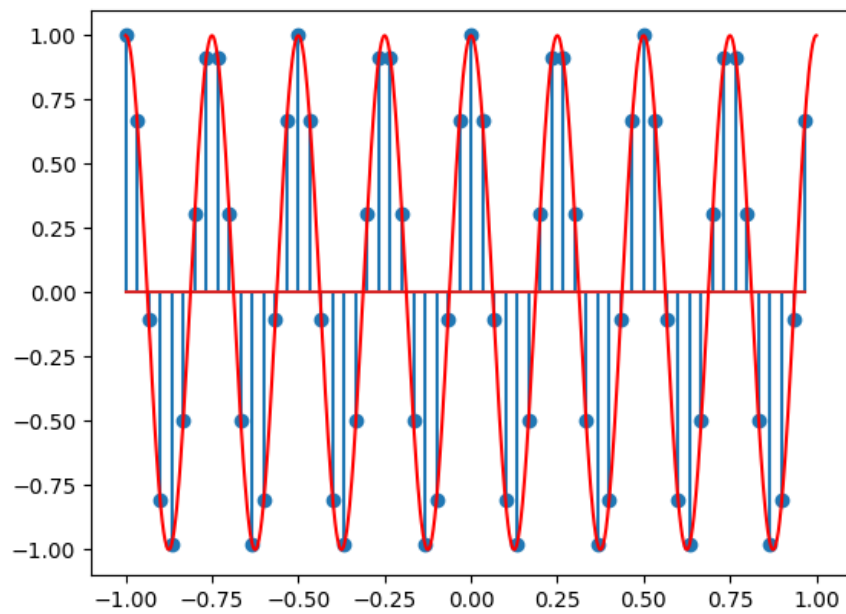


Figure-Sampling Signal

```
xmin = min(xt_sampled)
```

```
xmax = max(xt_sampled)
```

```
labels = 11
```

```
quantization_step = (xmax - xmin) / (labels - 1)
```

```
#include quantization logic here
```

```
quantized_signal=[]
```

```
for i in xt_sampled:
```

```
    value=round(i/quantization_step)*quantization_step
```

```
    quantized_signal.append(value)
```

```

quantized_signal=np.array(quantized_signal)

plt.figure()
plt.stem(pulse_train,quantized_signal)
#plt.plot(t, xt, 'r')

for label_value in np.arange(xmin, xmax + quantization_step, quantization_step):
    plt.axhline(y=label_value, color='g', linestyle='--')

plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.title("Quantized Signal with Quantization Labels")
plt.show()

```

Output:

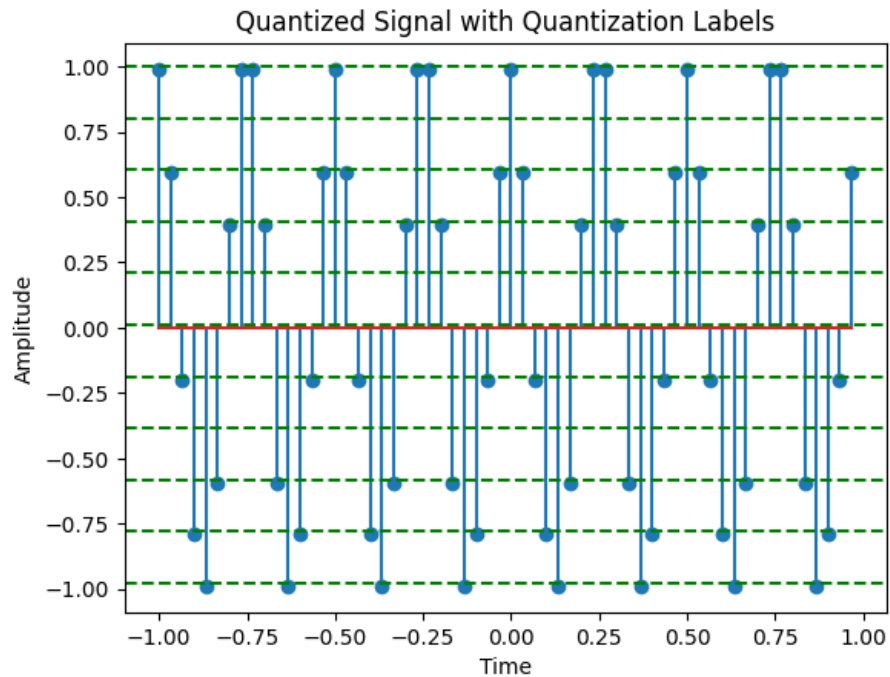


Figure-Quantized Signal

Discussion:

- Using Numpy and Matplotlib module I have implemented the ADC Conversion

b) Problem: Display Different kinds of unit signals

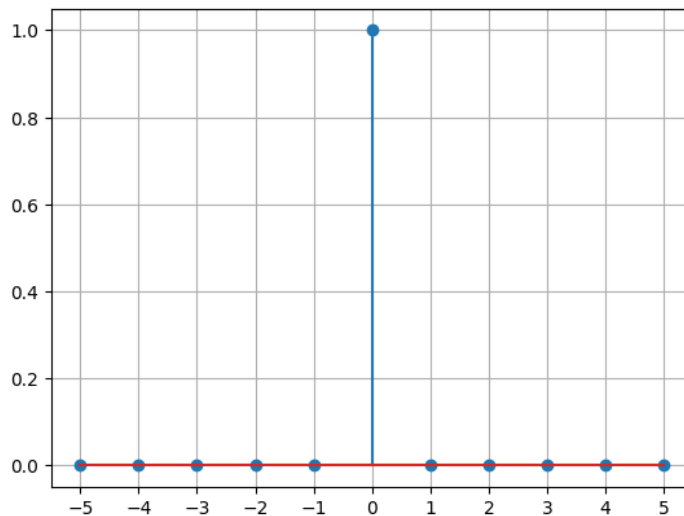
Logical Solution:

- Unit sample signal: when $n=0$ we get value 1.
- Unit sample signal: when $n \geq 0$ we get value 1.
- Unit ramp signal: when $n \geq 0$ we get value equal to n .

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt

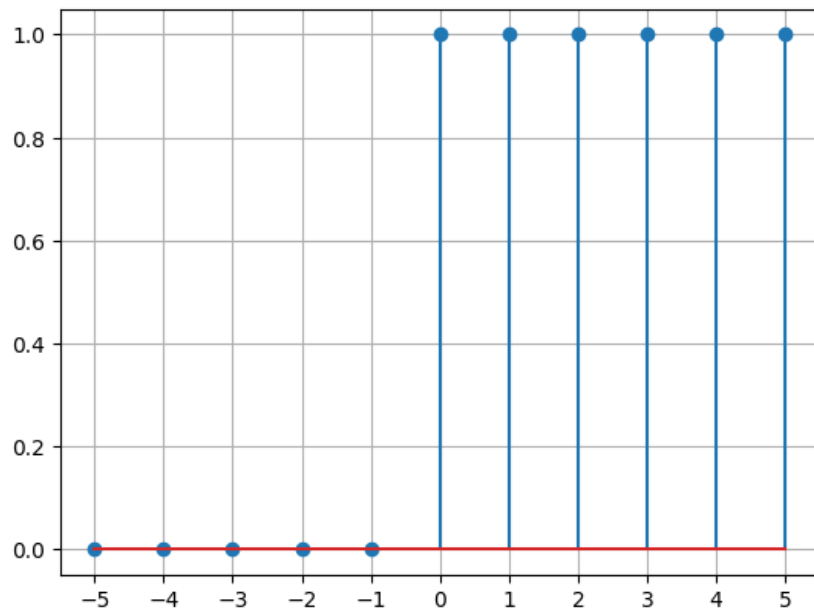
# Unit sample signal
n=np.array([i for i in range(-5,6,1)])
del_n=[]
for i in n:
    if i==0:
        del_n.append(1)
    else:
        del_n.append(0)
del_n=np.array(del_n)
plt.xticks(n)
plt.grid(True)
plt.stem(n,del_n)
plt.show()
```



Output-Unit Sample Signal

unit step signal

```
u=[]  
for i in n:  
    if i>=0:  
        u.append(1)  
    else:  
        u.append(0)  
del_n=np.array(del_n)  
plt.grid(True)  
plt.stem(n,u)  
plt.xticks(n)  
plt.show()
```

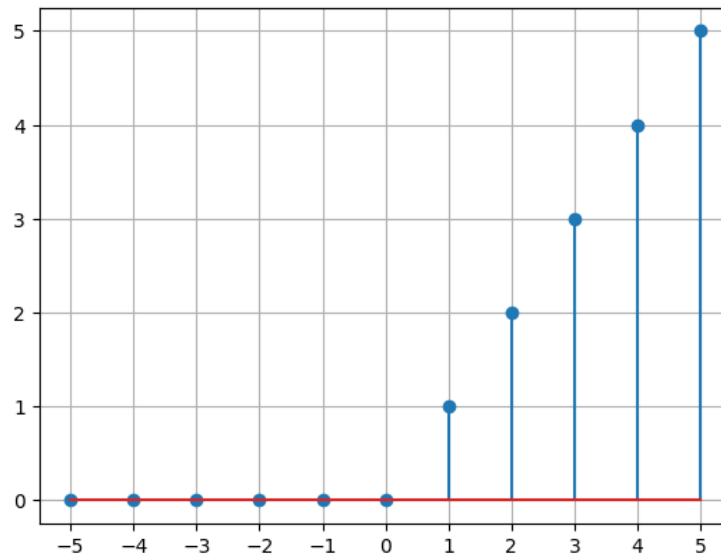


Output- Unit Step Signal

Unit Ramp signal

```
ur=[]  
for i in n:  
    if i>=0:  
        ur.append(i)  
    else:  
        ur.append(0)  
del_n=np.array(ur)  
plt.grid(True)
```

```
plt.stem(n,ur)
plt.xticks(n)
plt.show()
```



Output- Unit Ramp Signal

Discussion:

- Using matplotlib library I have figured out the signal types.

c) Problem: Different kinds of Signal Manipulation Technique

$$Y(n)=3X(-2n-2)$$

Logical Solution:

- Shifting- $y(n)=x(n-k)$ $k>0$: right shifting, $k<0$ left shifting
- Reversal- $y(n)=x(-n)$
- Amplitude Scaling- $y(n)=Ax(n)$
- Time Scaling- $y(n)=x(an)$

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt

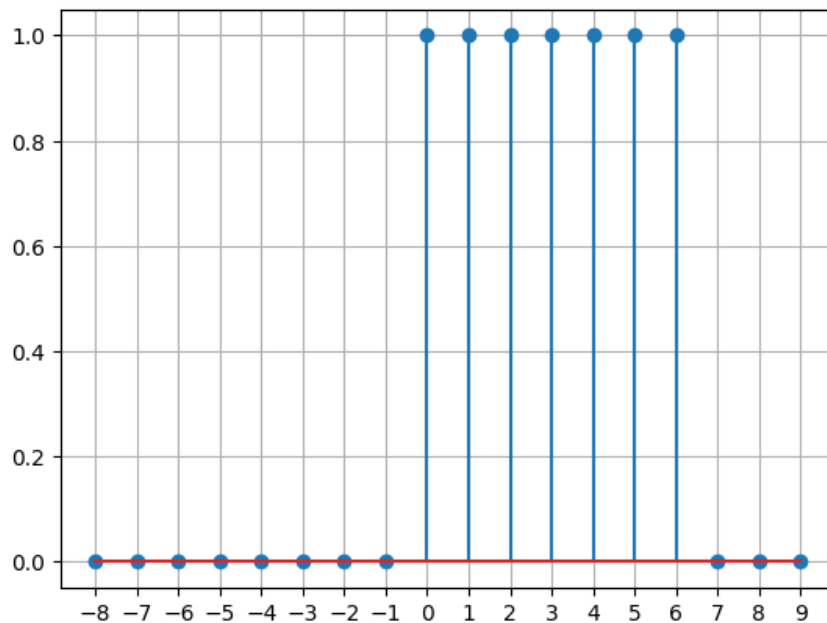
n=np.array([i for i in range(-8,10,1)])
signal=[]
```

```
def value(s):
    if s>=0 and s<=6 and round(s)==s:
        return 1
    return 0
```

```
signal=np.array(list(map(value,n)))
```

Original Signal

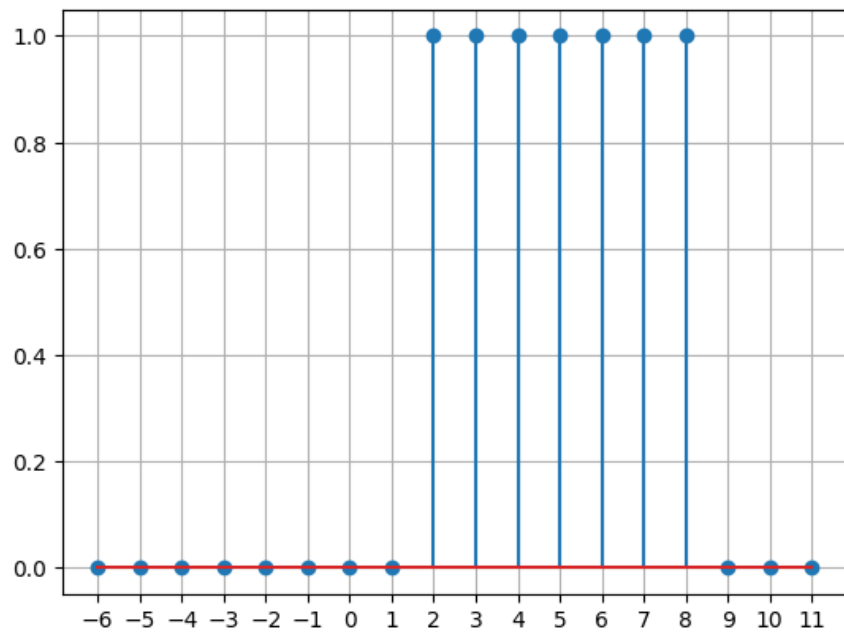
```
plt.stem(n,signal)
plt.xticks(n)
plt.grid(True)
plt.show()
```



Output- Original Signal

Shifting:

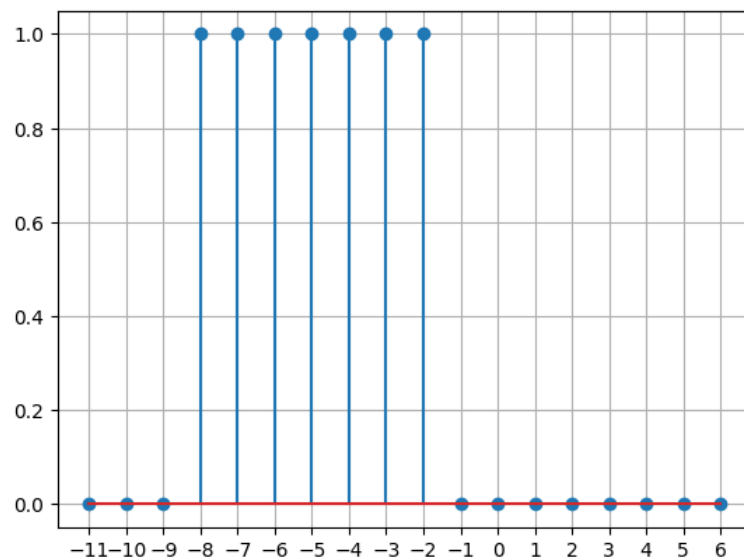
```
k=2
modulated=n+k
plt.stem(modulated,signal)
plt.xticks(modulated)
plt.grid(True)
plt.show()
```



Output- After Shifting

Reversal

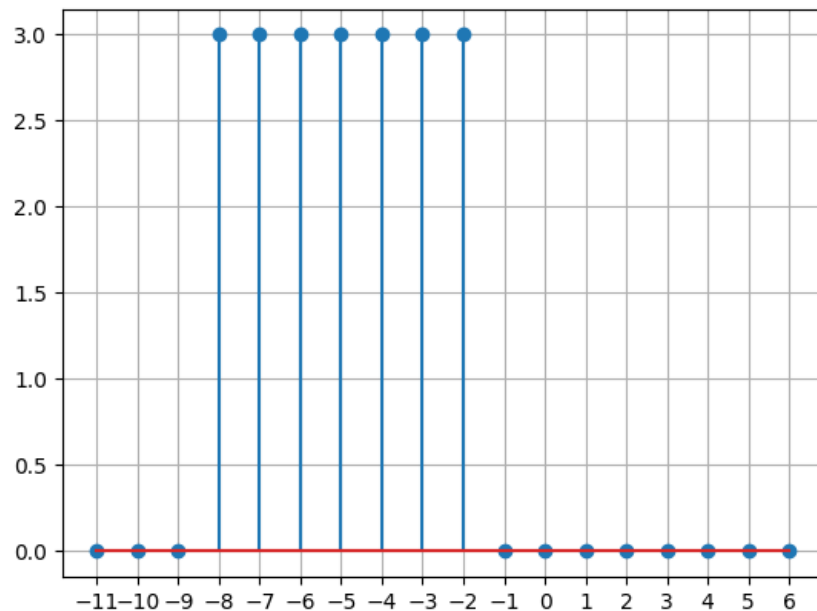
```
modulated=-modulated
plt.stem(modulated,signal)
plt.xticks(modulated)
plt.grid(True)
plt.show()
```



Output- After Reversing

Amplitude Scaling:

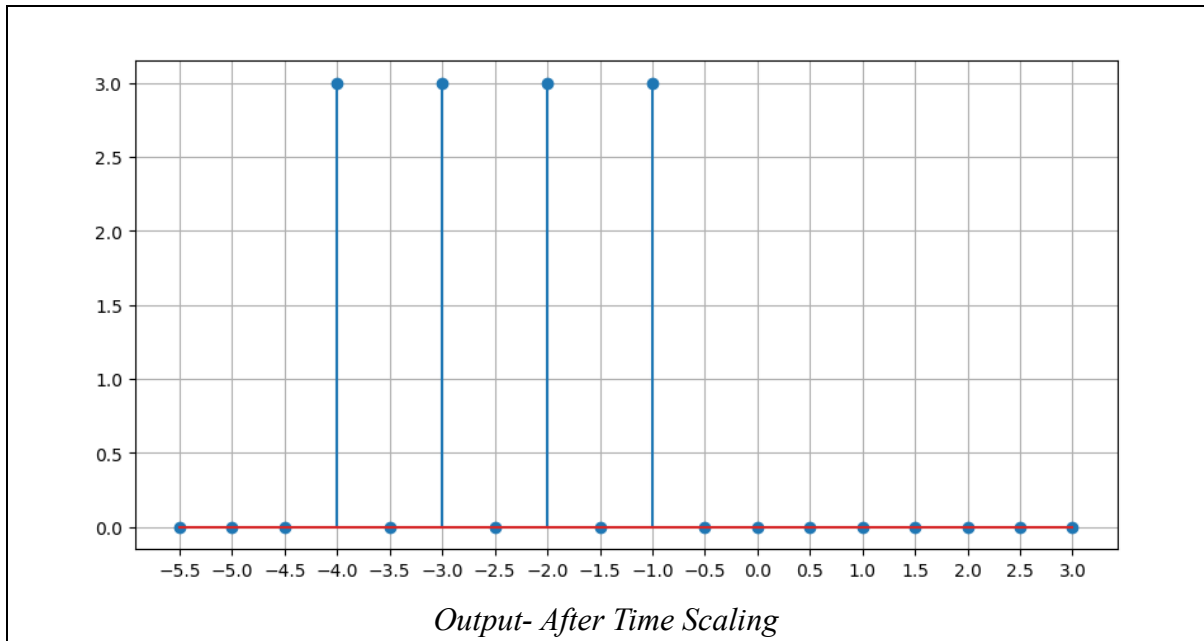
```
A=3  
signal=A*signal  
plt.stem(modulated,signal)  
plt.xticks(modulated)  
plt.grid(True)  
plt.show()
```



Output- After Amplitude Scaling

Time Scaling:

```
T=2  
def new_signal(x,y):  
    for i in range(len(x)):  
        if round(x[i])!=x[i]:  
            y[i]=0  
    return y  
modulated=modulated/T  
signal=new_signal(modulated,signal)  
plt.figure(figsize=(10,5))  
plt.stem(modulated,signal)  
plt.xticks(modulated)  
plt.grid(True)  
plt.show()
```



Discussion:

- As the order matters, I have completed shifting as 1st operation then remaining others.
- For signal manipulation shifting the 1st priority.